

Whole-Branch Independent Code Review — Session (main, ~33 commits)

Revision: 1 **Last modified:** 2026-07-10T00:00:00Z **Reviewer:** Stream Rev — independent, structurally separate from the authors of this session's work (§11.4.142 / §11.4.165 / §11.4.125). **Mode:** READ-ONLY static review; two read-only verifications run (`go vet ./pkg/challenge/...` on the challenges submodule — clean; targeted greps of Go JSON tags vs TS types). No files modified, no git writes, no contention with the server test stream.

Verdict

GO-WITH-FIXES. Critical: 0 · Important: 2 · Minor: 2.

The landed commits are correct and anti-bluff-sound. The two Important findings are (a) an anti-bluff *overclaim* in a test-only addition and (b) a disclosed release-readiness gap; neither is a shipped product regression and neither blocks the GO of this session's landed work. They must be resolved before the *frontend* is declared production-ready and before the memory-test's evidence claim is cited as leak-proof.

Commits / files reviewed (enumerated, §11.4.118)

Area	Commit(s)	Files reviewed
Server memory + fuzz gap- closure	f82a77e4	server/internal/api/memory_test.go, token_fuzz_test.go, EVIDENCE.md

Area	Commit(s)	Files reviewed
Token trust boundary (cross-check)	(existing)	server/internal/api/token.go (sign/Verify), handlers_artifact.go:resolvePublicKey
Challenges race + FAIL-bluff fix	submodule 5bac429 (ptr f553104f)	pkg/challenge/antibluff.go, pkg/challenge/result.go, pkg/challenge/base.go, tests/chaos/chaos_test.go
ota-manager type fixes 118 → 85	34f7dcf6	src/lib/api-client.ts, src/types/api.ts (vs Go wire.go, audit_wire.go)
shadcn WCAG-AA palette	a8c12d9a	CSS-token diff (stat-only: no code files touched)
llms_verifier RED fix ptr bump	5a3e036a	submodule pointer only
Hygiene sweep	74a417bc...HEAD	mutation-marker scan, force-push residue scan, EVIDENCE-artifact presence, dist tracking history, working-tree status

Findings (most-severe first)

IMPORTANT-1 (anti-bluff, §11.4/§11.4.107(10)) — memory_test self-calibrated threshold cannot detect a *steady linear* leak

server/internal/api/memory_test.go:172-189. $\text{ref} = \max(\text{growth1}, \text{growth2})$, $\text{threshold} = \text{ref} * 4$ (floor 512 KiB), FAIL if $\text{growth3} > \text{threshold}$. A genuine constant per-request leak retains $\sim$ equal bytes each equal-size batch, so $\text{growth1} \approx \text{growth2} \approx \text{growth3}$; the reference is *contaminated by the very leak it is trying to detect*, and $\text{growth3} (= L) < 4 \cdot L = \text{threshold} \rightarrow$ **PASS on a real leak**. The test only catches an *accelerating* leak or one exceeding the absolute floor.

- **Failure scenario:** a handler that leaks ~ 200 KiB/batch (≈ 137 B/req)
 $\rightarrow \text{ref} \approx 200$ KiB, $\text{threshold} = \max(800 \text{ KiB}, 512 \text{ KiB}) = 800 \text{ KiB}$,
 $\text{growth3} \approx 200 \text{ KiB} \rightarrow$ PASS, leak undetected.

- **Mitigation (why not Critical):** in the *observed* regime the post-GC noise ref is ~1–6 KB, so threshold collapses to the 512 KiB absolute floor → the test effectively asserts “batch C grew < ~512 KiB”, a real anchor that catches leaks above ~350 B/req/batch. It is also strictly net-additive (zero core-handler memory coverage existed before) and test-only (no product change).
- **Overclaim:** the commit message + EVIDENCE state it detects “a genuine per-request heap leak”; that is broader than what it guarantees. The §11.4.115 polarity proof (force threshold negative → FAIL) proves the *assertion wiring* fires, **not** that the calibration detects a leak — a golden-bad that is really a golden-noop.
- **Fix direction:** assert against an *absolute* per-request byte budget (or a cumulative $m_3 - m_0$ bound) instead of an in-band $4 \times \text{ref}$ derived from leak-contaminated batches; and soften the EVIDENCE wording to “catches leaks above the ~512 KiB/batch floor,” not “a genuine per-request leak.”

IMPORTANT-2 (release-readiness, disclosed) — ota-manager does not typecheck clean (85 router-cluster errors remain)

34f7dcf6 fixed 33 of 118 tsc errors; **85 remain** (router cluster). The ota-manager frontend therefore does not pass its own typecheck gate (94fb10a2 made that gate type-check source). This is **honestly disclosed** in the commit body and is not a bluff, but it means the ota-manager surface is not in a shippable state.

- **Not a correctness defect in what landed:** the *types added are genuine*, not tsc-silencing — `Artifact.os/target_model`, `artifact_id`, `AuditLogList.items/next_cursor`, `DeltaView` all verified byte-for-byte against the Go JSON contract (`server/internal/api/wire.go:44,111-127,160-161`; `audit_wire.go:35-36`). No any / as unknown / @ts-ignore introduced.
- **Fix direction:** finish the router-cluster fixes or formally park ota-manager (READINESS doc) before any tag that claims the frontend ships; keep it out of the release GO until 0 errors.

MINOR-1 (§11.4.30 hygiene, pre-existing) — tracked build artifacts under `clients/ota-manager/dist/`

`clients/ota-manager/dist/{index.html,assets/*.js,*.css}` are git-tracked build derivatives. History shows this predates the session (first tracked at 5b0ac61c “Auto-commit”), so it is **not this session’s**

regression, but the working tree currently shows dist churn (hashed bundles D/A). Fix direction: gitignore `clients/ota-manager/dist/` and untrack, or document a regen mechanism (§11.4.77); do it as its own commit.

MINOR-2 (§11.4.11 working-tree hygiene) — stray temp files + dirty submodule, uncommitted

Repo root holds stray `toPdfViaTempFile*-0.html / *-1.pdf` (pandoc/weasyprint temp spill); submodules/helixqa shows a modified (dirty) pointer; an in-flight `docs/qa/20260709-server-postgres-integration/EVIDENCE.md` is staged-but-uncommitted. None are landed defects — they are in-flight state — but they must be cleaned/committed before a release tag so the tree is quiescent (§11.4.84 / §11.4.121). The postgres EVIDENCE.md itself reads as substantive (real dual-run per-test timings, rootless-podman boot) and is not an obvious bluff, but it is not yet part of a reviewed commit.

Clean (checked, no finding) — anti-bluff coverage list

- **Challenges RecordAction mutex fix (5bac429) — CORRECT.** Lock wraps exactly the append; no blocking op inside the lock; nil-guard preserved. `Result` is used by pointer everywhere — `base.go:269` decodes into `var result Result` then returns `&result`; `CreateResult` returns `*Result` from a fresh composite literal. `go vet ./pkg/challenge/...` is clean → no copylocks violation. No fix-A-create-B. `encoding/json` ignores the unexported `mu` → serialized `Result` byte-unchanged.
- **Challenges chaos FAIL-bluff fix (5bac429) — CORRECT.** 500-NUL-byte path (EINVAL, always-fail-for-script-reason, §11.4.1) replaced with a valid long nested path + a NUL-path graceful-degrade subtest. Genuine defect, genuine fix.
- **FuzzTokenSignerVerify — faithful, not a tautology.** Targets the real `signer.Verify`; property asserted **only on accept** (garbage is expected to reject). Independent re-derivation (`hmac.New(sha256, secret)` over `parts[0]`) exactly matches `token.go:sign` (`mac.Write([]byte(encPayload))`); `base64url` + Claims-JSON re-decode + expiry re-check are all independent of `Verify`'s internals. Panic-freedom enforced by the fuzz runner.

- **Trust boundary intact.** `handlers_artifact.go:resolvePublicKey` returns the key only from `s.pubKey` (server config); there is no request path into it. No regression.
 - **shadcn/WCAG a8c12d9a** touches CSS tokens + evidence only — no `.ts/.tsx/.go/.js`, no auth/security surface.
 - **No mutation markers committed** across 74a417bc..HEAD (§11.4.84 clean).
 - **No force-push residue** in tracked `scripts/` (§11.4.113 clean).
 - **EVIDENCE artifacts present** for the testgap-closure run (md/html/pdf/docx).
-

Reasoning for GO-WITH-FIXES

Every landed source/test change is correct and genuinely exercises real behavior; the trust boundary, the concurrency fix, and the fuzz target are sound. The two Important findings are a test-only anti-bluff *overclaim* (mitigated by an absolute floor, net-additive) and a *disclosed* frontend typecheck gap — both tracked, neither a hidden shipped regression. Resolve IMPORTANT-1's wording/threshold and gate the ota-manager frontend on 0 typecheck errors before a release that includes it; clean the working tree before tagging.